

some other journal library tomorrow, the code in the *UserRepositoryJournalProxy* also needs to be changed. We already know that a change in provider must not result in a change in consumer.

The Gang of Four suggested keeping a protection layer between our modules and the third party modules that are vulnerable to changes. The pattern to follow is ‘adapter’.

Adapter

The job of an adapter is to sit between the consumer and the third party provider. It offers a known and consistent interface to the consumers irrespective of the changes in the third party provider. The pattern can be depicted as shown in Figure 3, for the UMS.

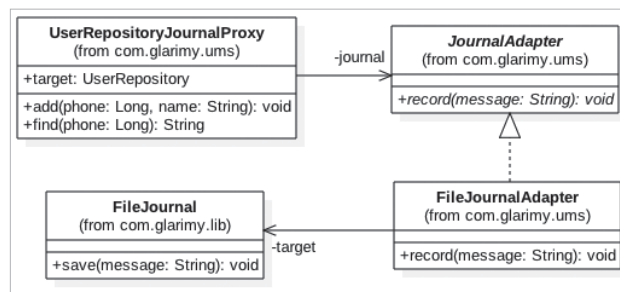


Figure 3: Adapter pattern

The notable observation here is that the *JournalAdapter* is an interface that offers a consistent function named *record()*.

```
package com.glarimy.ums;

public interface JournalAdapter {
    public void record(String message);
}
```

This interface is to be defined by the consumer according to the need. In our case, the consumer is *UserRepositoryJournalProxy*.

For different third party modules, there will be different implementations of *JournalAdapter*. Here is an implementation that uses the *FileJournal*.

```
package com.glarimy.ums;

import com.glarimy.lib.FileJournal;

public class FileJournalAdapter implements JournalAdapter {
    private FileJournal journal;

    public FileJournalAdapter() throws Exception {
        journal = new FileJournal();
    }
}
```

```
public void record(String message) {
    journal.save(message);
}
}
```

It can be observed that the *FileJournalAdapter* is hiding *FileJournal* from its consumers. A consumer, like *UserRepositoryJournalProxy*, refers only to *JournalAdapter*.

```
private JournalAdapter adapter;
```

The consumer uses a factory to load the adapter:

```
Factory factory = new ObjectFactory("config.properties");
this.adapter = (JournalAdapter) factory.get("adapter");
```

Our existing *ObjectFactory* works perfectly with the entry in the configuration file, which looks like this:

```
adapter=com.glarimy.ums.FileJournalAdapter
```

And the following will be the code in *UserRepositoryJournalProxy* that uses the adapter. Notice that the consumer is only referring to the interface of the adapter, not the third party module, directly.

```
public void add(Long phone, String name) {
    adapter.record("c=repository, s=add, a=" + "&" + name);
    target.add(phone, name);
    adapter.record("c=repository, s=add, r=void");
}

public String find(Long phone) {
    adapter.record("c=repository, s=find, a=" + phone);
    String name = target.find(phone);
    adapter.record("c=repository, s=find, r=" + name);
    return name;
}
```

With the help of an adapter interface and an able factory, we can plug in any other third party library into the system, without making changes to the consumer. We only will have different implementations of the adapter interface for different third party modules.

Again, this is a good time to summarise the adapter pattern.

1. An adapter acts as an intermediary between the consumer and a third party library.
2. An adapter offers a consistent interface to the consumers.
3. It translates the calls from the consumers to a specific third party library.